

Разработка игрового приложения «Коридор» с модулем стратегического поведения соперника

Александр Олегович Карпунин

Научный руководитель: А.В. Николаев

2026



Цель работы

Целью выпускной квалификационной работы является разработка игрового приложения «Коридор», включающего в себя реализацию правил игры, визуализацию игрового поля и модуль стратегического поведения, объединяющий четыре алгоритма программного соперника.



Задачи работы

Основные задачи

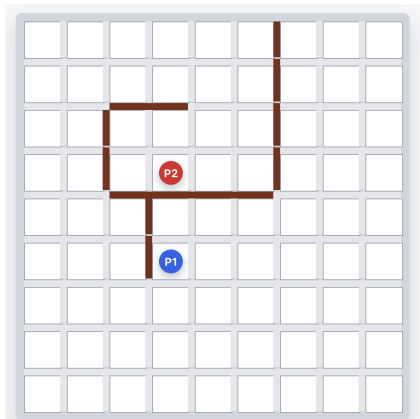
- 1 изучить правила игры «Коридор»;
- 2 реализовать алгоритм проверки корректности перемещения фишки;
- 3 реализовать алгоритм проверки корректности установки перегородки с условием достижимости целевых сторон поля;
- 4 реализовать алгоритм поиска кратчайшего пути до целевой стороны поля на основе обхода графа в ширину;
- 5 разработать функцию оценки позиции игрока с настраиваемыми весами и механизмом их коррекции по результатам партий;
- 6 реализовать четыре алгоритма программного соперника: случайный выбор хода, Минимакс, альфа-бета отсечение и Монте-Карло;
- 7 провести сравнение алгоритмов в серии автоматических партий;
- 8 разработать интерфейс игрового приложения.



Игра «Коридор»

Правила

- цель игрока — довести свою фишку до противоположной стороны игрового поля быстрее соперника;
- за ход можно передвинуть свою фишку или поставить стену, которая будет блокировать перемещение между соседними клетками;
- установленная стена не должна препятствовать достижению игроками их победных сторон поля;
- количество стен ограничено.



Архитектура приложения



Генерация допустимых ходов

Перемещение фишки

Для текущей позиции строится множество доступных соседних клеток с учетом стен, границ поля, прыжков через соперника и диагональных прыжков при невозможности прямого прыжка.

Установка стены

Для стены проверяются пересечения, наличие стен у игрока и сохранение пути к целевой стороне для обоих игроков. Достижимость проверяется обходом графа в ширину.



Общая форма

$$F(s) = \sum_i \lambda_i f_i(s)$$

Используемые признаки

- разность длин кратчайших путей игроков;
- мобильность фишек;
- оставшиеся стены;
- прогресс продвижения к целевой стороне поля.



Обучение весов функции оценки

Веса функции оценки вынесены в отдельный файл и подгружаются при старте игрового приложения.

После обучающих партий веса корректируются и сохраняются между запусками приложения.

Сравнение весов

Был проведен эксперимент: AlphaBeta с начальными весами играл против AlphaBeta с обновлёнными 50 партий на поле 11×11 со сложностью HARD. Первый проиграл со счетом 19:27, а еще 4 партии закончились ничьей.



Случайный выбор

MiniMax

AlphaBeta

MonteCarlo

Настройка сложности

Для каждого алгоритма предусмотрены три уровня сложности. Уровень определяет допустимую глубину поиска и ограничение времени на выбор хода.



```
minimax(state, depth, maximizing):  
    if terminal(state) or depth == 0: return evaluate(state)  
    moves = sortedLegalMoves(state)  
    if maximizing:  
        best = -inf  
        for move in moves:  
            child = apply(state, move)  
            best = max(best, minimax(child, depth - 1, false))  
    else:  
        best = +inf  
        for move in moves:  
            child = apply(state, move)  
            best = min(best, minimax(child, depth - 1, true))  
    return best
```



```
alphabeta(state, depth, alpha, beta, maximizing):
    entry = transpositionTable[state]
    if entry is valid for depth: return entry.value
    if terminal(state) or depth = 0: return evaluate(state)
    moves = orderedLegalMoves(state)
    if maximizing:
        value = -inf
        for move in moves:
            child = apply(state, move)
            value = max(value, alphabeta(child, depth - 1, alpha, beta, false))
            alpha = max(alpha, value)
            if alpha >= beta: break
    else:
        value = +inf; обновить beta аналогично
    save(state, depth, value, boundFlag, bestMove)
    return value
```



```
monteCarloMove(state, timeLimit):  
    moves = sortedLegalMoves(state)  
    for move in moves: stats[move] = (score=0, visits=0)  
    while time remains:  
        move = selectByUCT(moves, stats)  
        next = apply(state, move)  
        result = rolloutWithHeuristics(next)  
        stats[move].visits += 1  
        stats[move].score += result  
    return move with maximal average score  
  
UCT = average + C * sqrt(log(totalVisits) / visits)
```



Игровой экран

Коридор

P1

Ходит игрок

P1: 4

P2: 9

Случайный

MiniMax

AlphaBeta

MonteCarlo

AlphaBeta: P2 стена (5, 3) - (6, 3)

Оценка	Время
-921	1545 мс
Глубина	Узлы
6	5364
Кандидаты	Отсечения
10383	990
TT Nix	

Статистика

Ходов: 21

Время игры (с): 54

Статус: партия идет

Подсказка хода

Начать сначала

История ходов:

1. Игрок P2 перешел на клетку (2, 5)
2. Игрок P1 перешел на клетку (8, 5)
3. Игрок P2 перешел на клетку (3, 5)
4. Игрок P1 перешел на клетку (4, 5)
5. Игрок P2 перешел на клетку (4, 5)
6. Игрок P1 поставил стену (4, 5) - (4, 6)
7. Игрок P2 перешел на клетку (4, 4)
8. Игрок P1 поставил стену (4, 3) - (4, 4)
9. Игрок P2 перешел на клетку (4, 5)
10. Игрок P1 перешел на клетку (6, 5)
11. Игрок P2 перешел на клетку (4, 6)
12. Игрок P1 поставил стену (3, 6) - (4, 6)
13. Игрок P2 перешел на клетку (4, 5)
14. Игрок P1 поставил стену (1, 6) - (2, 6)
15. Игрок P2 перешел на клетку (3, 5)
16. Игрок P1 поставил стену (3, 2) - (4, 2)
17. Игрок P2 перешел на клетку (3, 4)
18. Игрок P1 поставил стену (2, 3) - (2, 4)
19. Игрок P2 перешел на клетку (4, 4)
20. Игрок P1 перешел на клетку (6, 4)
21. Игрок P2 поставил стену (5, 3) - (6, 3)

←

→

К текущему



Подсказки и объяснение хода

Коридор

P1

Ходит игрок

P1: 7

P2: 7

Случайный

MiniMax

AlphaBeta

MonteCarlo

AlphaBeta: P2 стена (3, 5) - (3, 6)

Оценка
-393

Время
3601 мс

Глубина
6

Узлы
11631

Кандидаты
25626

Отсечения
2001

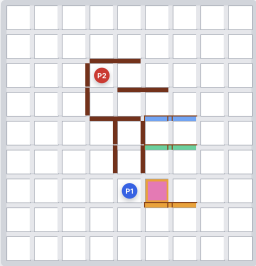
TT: 0%

Статистика

Ходов: 15

Время игры (с): 60

Статус: партия идет



Подсказка хода

Начать сначала

История ходов:

1. Игрок P2 перешел на клетку (2, 5)
2. Игрок P1 перешел на клетку (8, 5)
3. Игрок P2 перешел на клетку (3, 5)
4. Игрок P1 перешел на клетку (7, 5)
5. Игрок P2 перешел на клетку (4, 5)
6. Игрок P1 поставил стену (4, 4) - (4, 5)
7. Игрок P2 перешел на клетку (4, 4)
8. Игрок P1 поставил стену (3, 3) - (4, 3)
9. Игрок P2 перешел на клетку (3, 4)
10. Игрок P1 перешел на клетку (6, 5)
11. Игрок P2 поставил стену (5, 5) - (6, 5)
12. Игрок P1 поставил стену (2, 4) - (2, 5)
13. Игрок P2 поставил стену (5, 4) - (6, 4)
14. Игрок P1 перешел на клетку (7, 5)
15. Игрок P2 поставил стену (3, 5) - (3, 6)

←

→

К текущему

Александр Олегович Карпунин

Игровое приложение «Коридор»

14 / 25

Режимы автоматического сравнения

Турнир ИИ

ИИ P1	ИИ P2	Сложность P1	Сложность P2
Minimax	AlphaBeta	Средний	Легкий

Размер поля:

Количество партий:

[Запустить турнир](#) [Назад](#)

Спорт турниры: Minimax vs AlphaBeta
Поль: Больше поле 18 на 18, P2 сложности: Средний, P2 сложности: Легкий, партий: 6
Партия 1/6: начинает P2, после нас останется партий: 5
Партия 1/6, код 1/388, да гинета партии: 298, после партии останется партий: 5, последний ход: P2
Партия 1/6, код 2/388, да гинета партии: 298, после партии останется партий: 5, последний ход: P2
Партия 1/6, код 3/388, да гинета партии: 297, после партии останется партий: 5, последний ход: P1
Партия 1/6, код 4/388, да гинета партии: 296, после партии останется партий: 5, последний ход: P2
Партия 1/6, код 5/388, да гинета партии: 295, после партии останется партий: 5, последний ход: P1

Сравнение алгоритмов

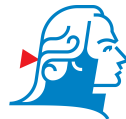
Сложность Random	Сложность Minimax	Сложность MonteCarlo	Сложность AlphaBeta	Размер поля
Средний	Средний	Средний	Средний	8

Партий на игру:

[Сравнить](#) [Назад](#)

Сравнение 1/6: Случайный vs Minimax
Сравнение 2/6: Случайный vs MonteCarlo
Сравнение 3/6: Случайный vs AlphaBeta
Сравнение 4/6: Minimax vs MonteCarlo
Сравнение 5/6: Minimax vs AlphaBeta

В приложении реализованы режимы турнира и сравнения алгоритмов с логами прогресса и итоговой статистикой.



Постановка эксперимента

Для каждой пары алгоритмов проводились 50 партий на полях размеров 7×7 , 9×9 , 11×11 . Была выбрана максимальная сложность каждого из них.

- сравнивались четыре алгоритма;
- использовались поля 7×7 , 9×9 , 11×11 ;
- для каждой пары алгоритмов проводились автоматические партии;
- фиксировались победы, длина партии, время хода, глубина и число просмотренных состояний.



Результаты: поле 7×7

Алгоритм	Партий	Победы	Поражения	Лимит	Доля побед
MiniMax	150	126	24	0	84,00%
AlphaBeta	150	93	57	0	62,00%
MonteCarlo	150	81	69	0	54,00%
Случайный	150	0	150	0	0,00%



Результаты: поле 9×9

Алгоритм	Партий	Победы	Поражения	Лимит	Доля побед
MonteCarlo	150	107	43	0	71,33%
MiniMax	150	97	52	1	64,67%
AlphaBeta	150	94	55	1	62,67%
Случайный	150	0	148	2	0,00%

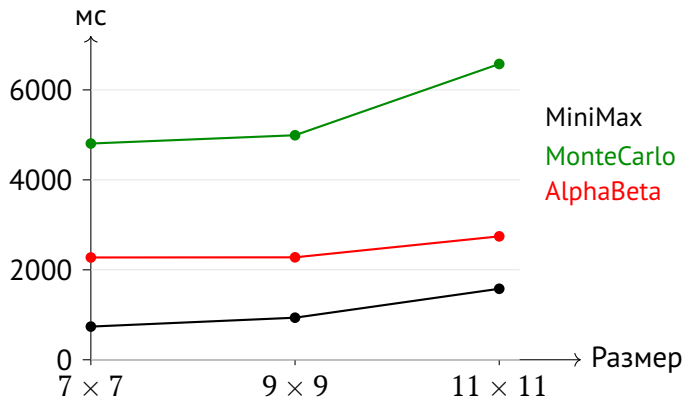


Результаты: поле 11×11

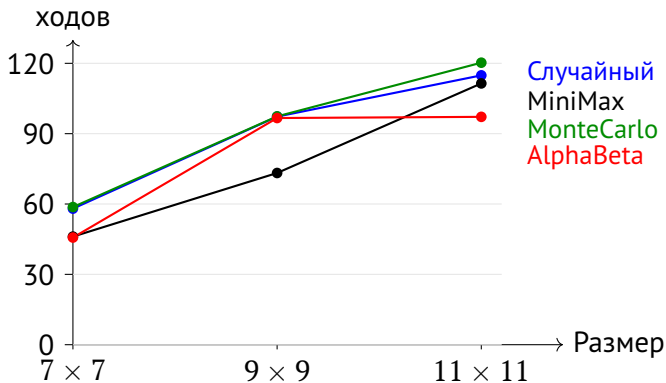
Алгоритм	Партий	Победы	Поражения	Лимит	Доля побед
AlphaBeta	150	121	27	2	80,67%
MonteCarlo	150	88	59	3	58,67%
MiniMax	150	84	62	4	56,00%
Случайный	150	0	145	5	0,00%



Среднее время выбора хода



Средняя длина партии



Результаты работы

- разработано игровое приложение «Коридор»;
- реализованы правила игры и проверка достижимости целей;
- реализованы четыре алгоритма программного соперника;
- добавлены режимы игры против ИИ, ИИ против ИИ, турнира, сравнения и подсказки;
- проведено экспериментальное сравнение алгоритмов, в рамках которого наиболее эффективным себя показал AlphaBeta.





Russell, S., Norvig, P.

Artificial Intelligence: A Modern Approach. — Pearson, 2021.



Berlekamp, E.R., Conway, J.H., Guy, R.K.

Winning Ways for Your Mathematical Plays. — A K Peters, 2001.



Massague Respoll, V., Brown, J.A., Aslam, H.

Monte Carlo Tree Search for Quoridor. — 2018.



Moskowitz, B., Joseph, J.

Quoridor-AI. — 2022.



Спасибо за внимание!



Архивы приложения для Windows и macOS

[https://github.com/karpuninaleksandr/quoridor/
releases](https://github.com/karpuninaleksandr/quoridor/releases)

В релизах доступны архивы с готовой сборкой приложения.

